

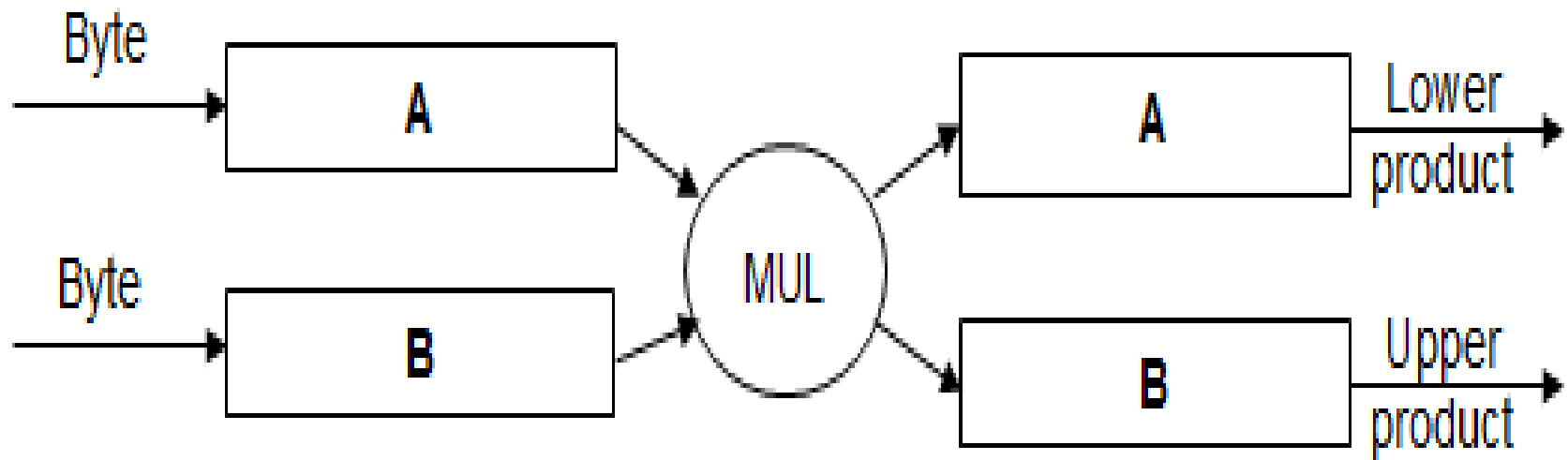
MICROPROCESSOR BASED SYSTEMS

CMP 410

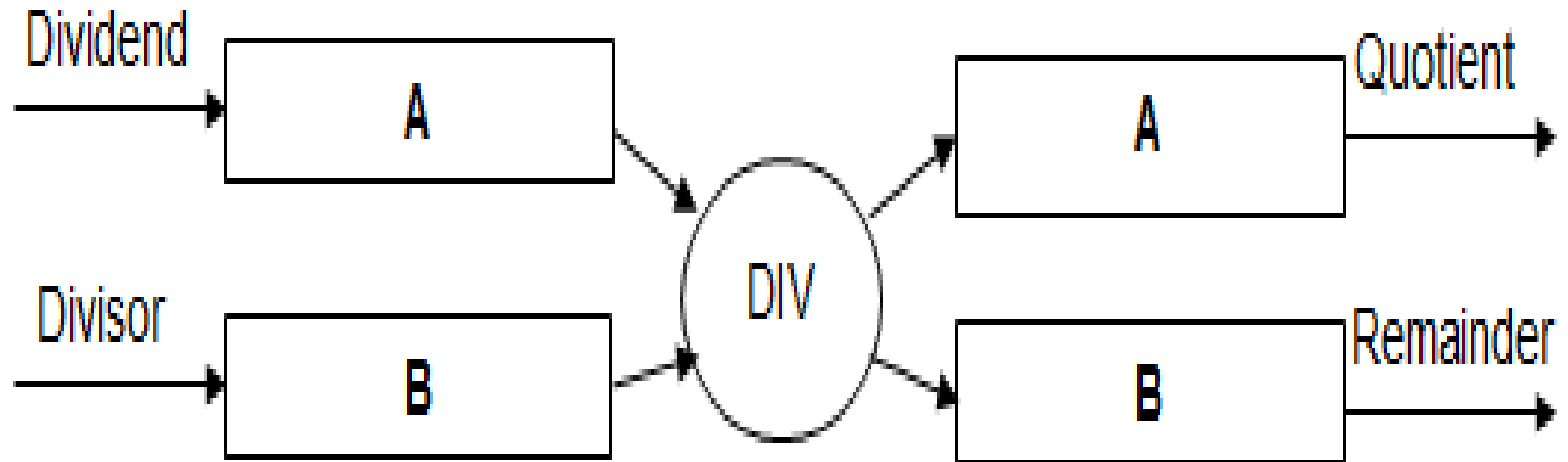
LECTURE 5

<https://AssemSite8.wix.com/site8>

Executing the “MUL” instruction in MCS-51

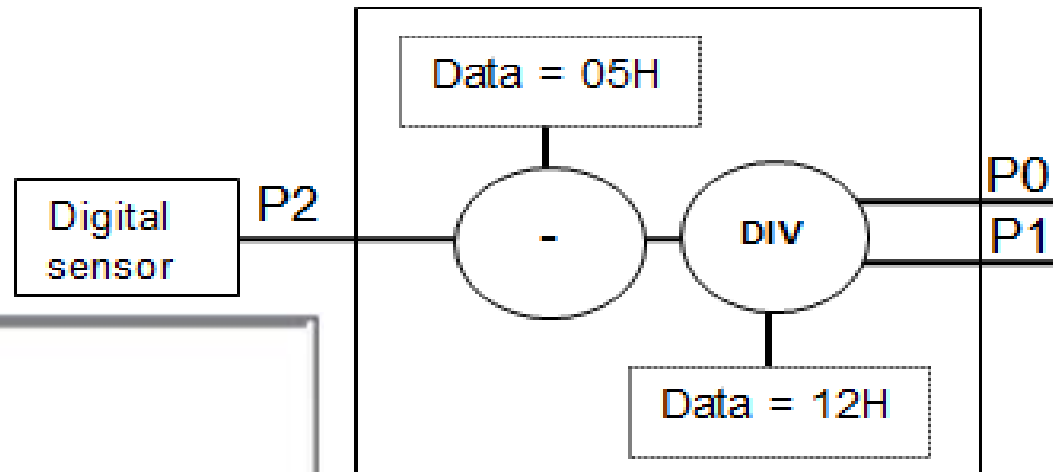


Executing the “DIV” instruction in MCS-51



EXAMPLE

Write assembly code based on MCS-51 to perform the arithmetic operations for the following block diagram.



```
INCLUDE 8051.mc
```

```
MOV P2, #FFH ; make P2 as input port
```

```
MOV A, P2 ; Read p2 to A
```

```
CLR C ; clear carry
```

```
SUBB A, #05H ; Subtract 5 from A
```

```
MOV B, #12H ; load 12h to reg-B
```

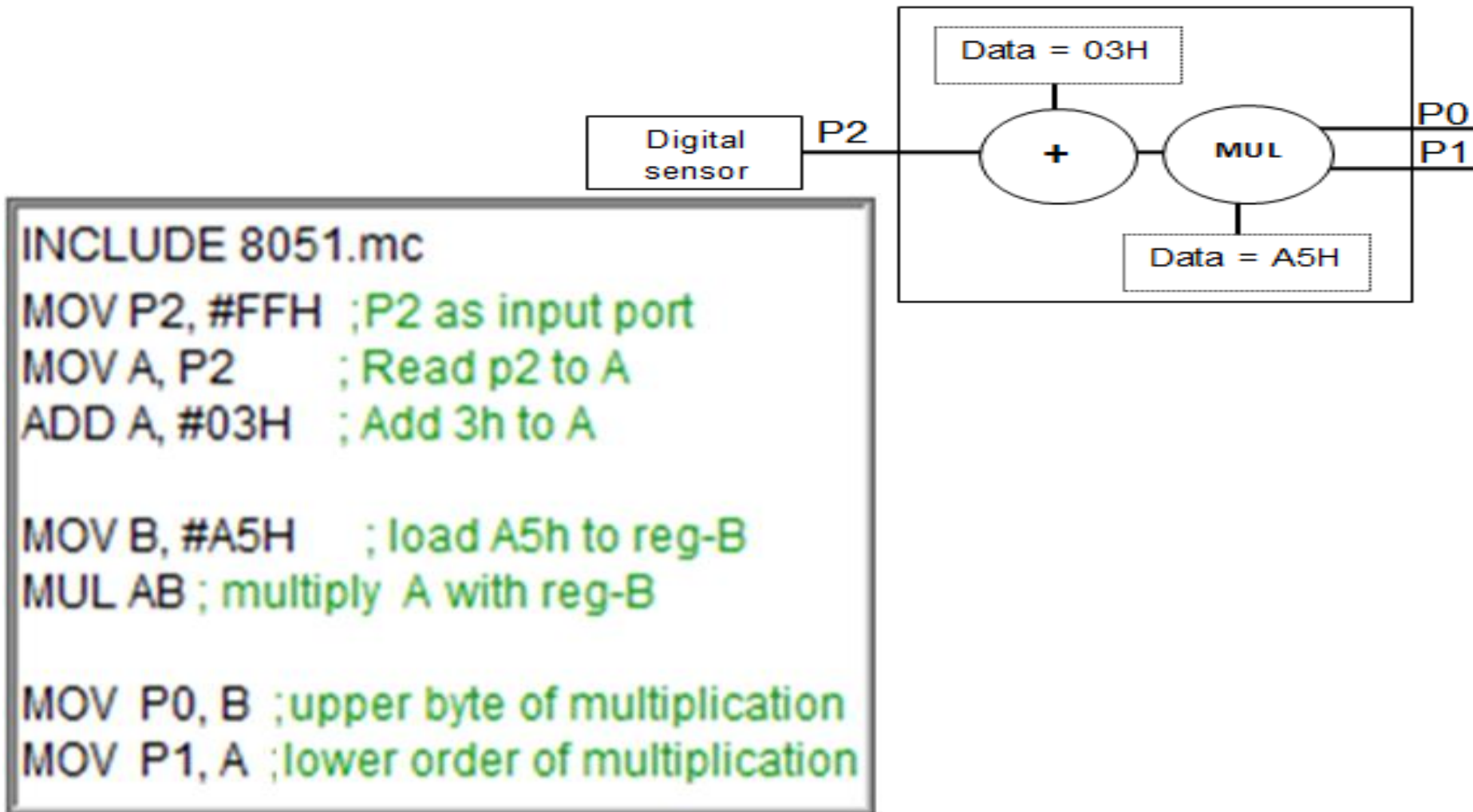
```
DIV AB ; divide A by reg-B
```

```
MOV P0, B ;Write Remainder to P0
```

```
MOV P1, A ;Write Quotient to P1
```

EXAMPLE

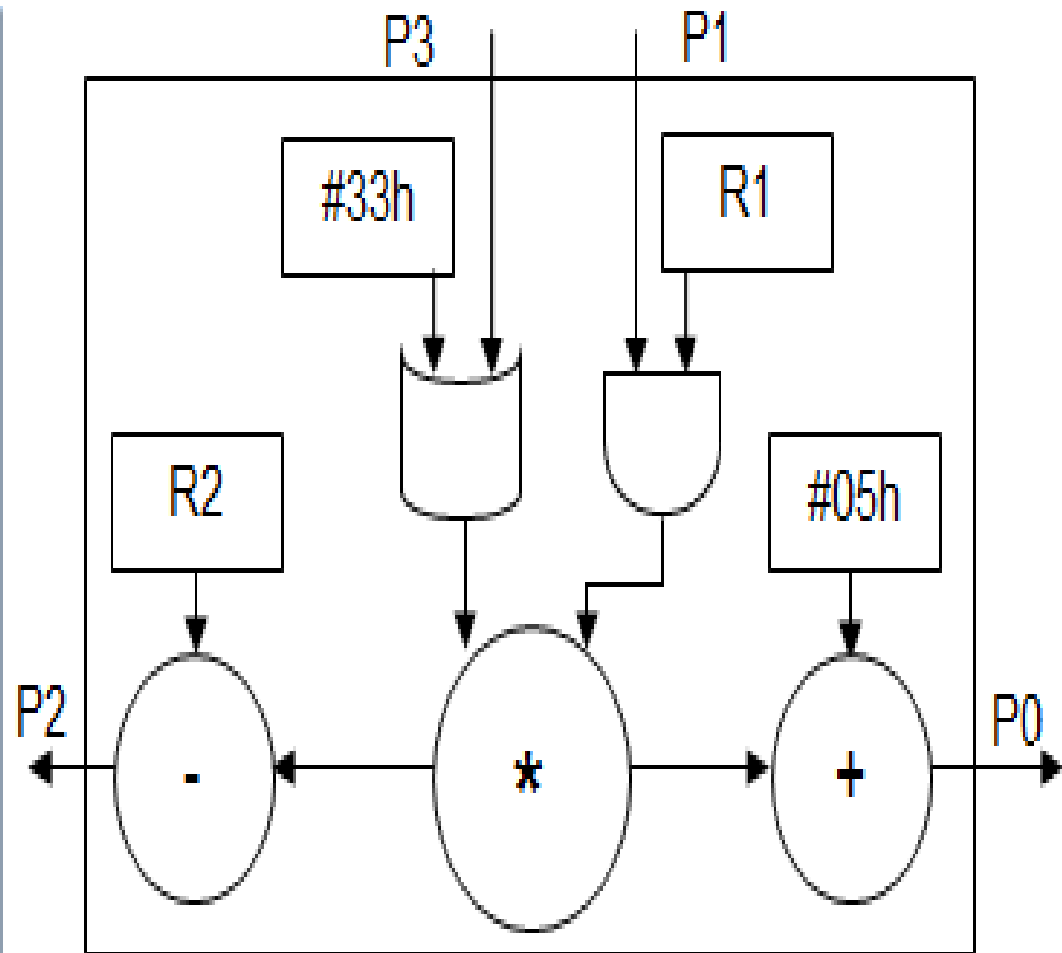
Write assembly code based on MCS-51 to carry out the arithmetic operations for the following block diagram.



EXAMPLE

Write assembly code based on MCS-51 to achieve the operations for the following block diagram.

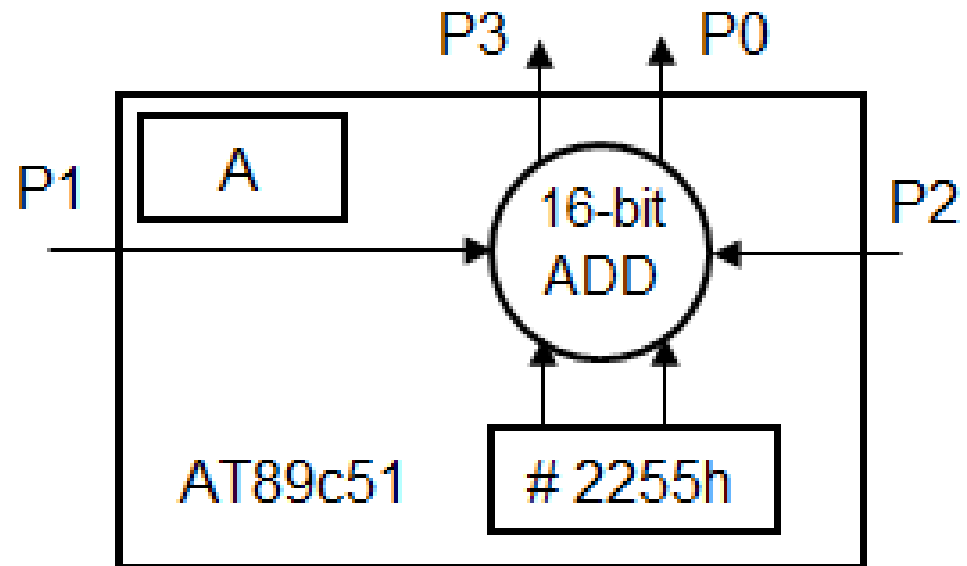
```
ORG 0H
MOV P1,#FFH
MOV P3,#FFH
MOV A, P1
ANL A, R1
MOV B, P3
ORL B, #33H
MUL AB
ADD A, #5H
MOV P0, A
MOV A, B
CLR C
SUBB A, R2
MOV P2, A
```



EXAMPLE

Write subprogram to add two 16-bit numbers for the following diagram (keep the data on "A" without change at the end of the program).

```
ORG 0
MOV P1,#0FFH
MOV P2,#0FFH
Push A ;Store byte
.....
MOV A,P2
ADD A,#55H
MOV P0,A ;--
MOV A,P1
ADDC A,#22H
MOV P3,A
Pop A ;retrieve byte
SJMP $
```



Decimal Adjust (DA A) Instruction

➤ If $A_{0 \rightarrow 3} > 9$ **or** $AC = 1$ then $A_{0 \rightarrow 3} + 6h$

➤ If $A_{4 \rightarrow 7} > 9$ **or** $CY = 1$ then $A_{4 \rightarrow 7} + 6h$

If A has value **DCh** get the BCD value
answer

$Ch + 6h = 12h \rightarrow$ **carry** = 1 and $A_{0 \rightarrow 3} = 2$

$(Dh + \text{carry} = 1) + 6 = 14h \rightarrow$ **CY** = 1 and $A_{4 \rightarrow 7} = 4$

DA(DCh) = 42

A has value **9Ah** get the BCD value
answer

$Ah + 6h = 10h \rightarrow \text{carry} = 1$ and $A_{0 \rightarrow 3} = 0$

$(9h + \text{carry} = 1) + 6 = 10h \rightarrow \text{CY} = 1$ and $A_{4 \rightarrow 7} = 0$

DA(A) = 00

A has value **12h** and **CY=AC=1** get the BCD value

answer

Ac=1 then

$2h + 6h = 08h \rightarrow \text{carry} = 0$ and $A_{0 \rightarrow 3} = 8$

CY=1 then

$(1h + \text{carry} = 0) + 6 = 07h \rightarrow \text{CY} = 0$ and $A_{4 \rightarrow 7} = 7$

DA(A) = 78

EXAMPLE

A digital circuit based on MCS-51 is designed for adding 2 **BCD-numbers** that received from P1 and P2.

- ❑ **Write** assembly program to display the summation of these 2 data on 7-segments via any available port(s).
- ❑ If the 2 **BCD-numbers** were **81h** and **70h**, what is the output BCD-number on its **7-segments**.

The assembly code

```
INCLUDE 8051.mc

MOV P1, #FFH    ;P1 as input port
MOV P2, #FFH    ;P2 as input port

MOV A, P1       ;Read p1
MOV B, P2       ;Read p2

ADD A, B        ; B + A
DA A            ;Decimal adjusting

MOV P0, A       ;out lower 2 digits
MOV P3.7, C     ;out upper digits
```

	Binary								HEX
A	1	0	0	0	0	0	0	1	81h
+									
B	0	1	1	1	0	0	0	0	70h
A	1	1	1	1	0	0	0	1	F1h

ACC. has value **F1h** , **AC=Cy= '0'**

answer

1h+0 = 01h → **carry=0** and $A_{0 \rightarrow 3} = \textcircled{1}$

(Fh+ **carry=0**)+6=15h → **CY =1** and $A_{4 \rightarrow 7} = \textcircled{5}$

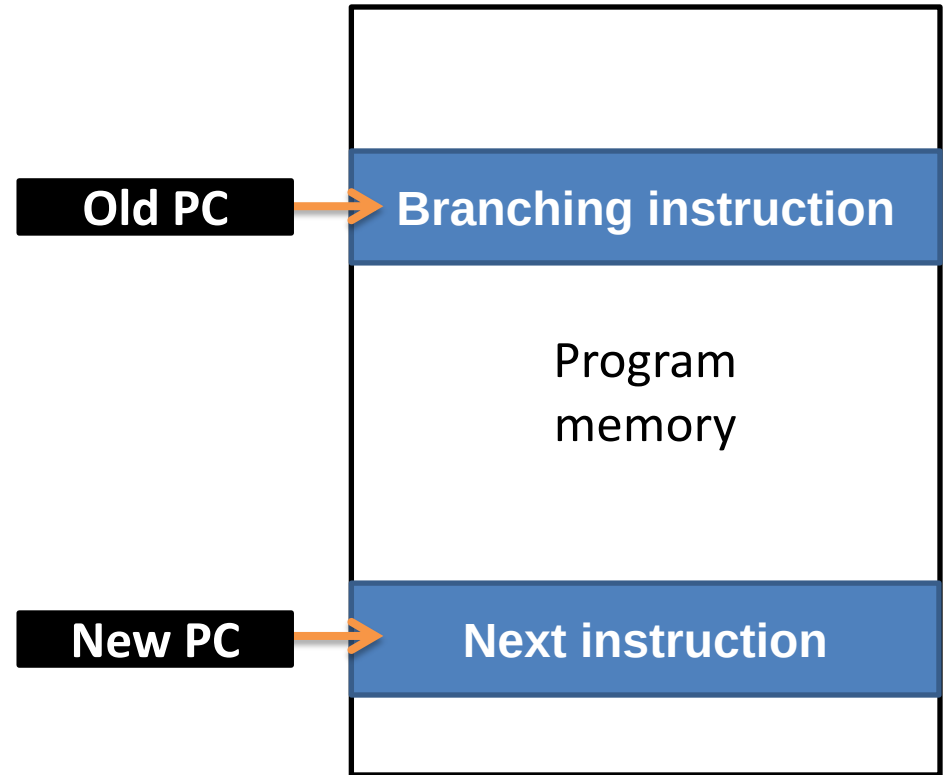
DA(A)= **51**

The total value = **151**



The Branching instructions of MCS-51

Instructions that responsible for changing the content of **PC** to follow the subsequent instruction in the program memory.



The **un**conditional branching instruction

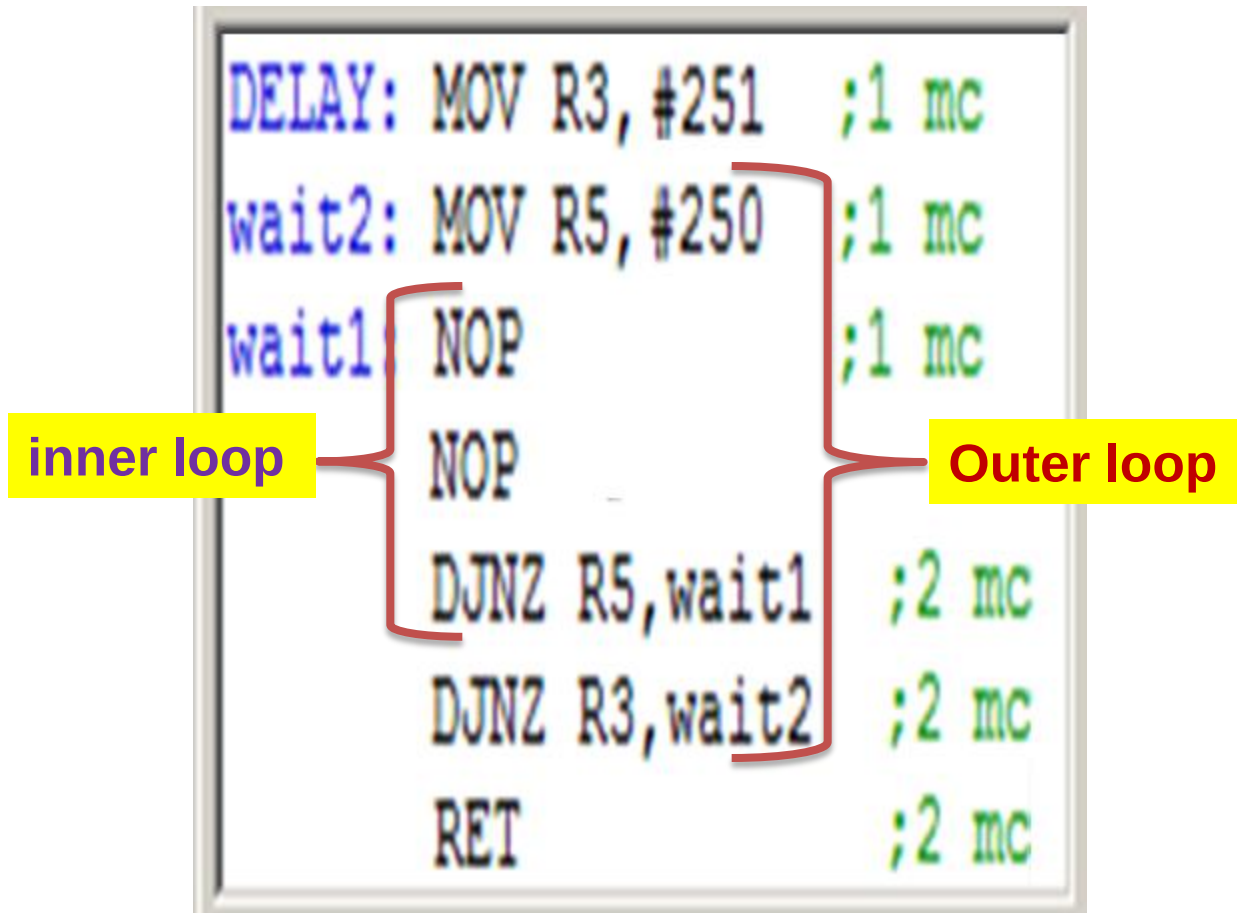
Unconditional Branching Instructions		
Instruction	Description	Branching
L JMP addr16	long jump	64 KB
A JMP addr11	absolute jump	2KB
S JMP rel	short jump	Relative to PC
L CALL addr16	long call	64 KB
A CALL addr11	absolute call	2KB
RET	Return from the calling	64 KB
RETI	Return from the interrupt service	64 KB

Relative conditional Branching Instructions

Instruction	Condition	
JZ rel	Jump if Accumulator	= 0h
JNZ rel		≠ 0h
JC rel	Jump if carry flag	= 1
JNC rel		≠ 1
JB Bit, rel	Jump if addressable bit	= 1
JNB Bit, rel		≠ 1
JBC Bit, rel		= 1 and clear it
DJNZ Rx, rel	Dec Rx and jump	Rx ≠ 0
DJNZ Rn, rel	Dec Rn and jump	RN ≠ 0
CJNE Rn, #data, rel	Compare and jump	if Rn ≠ data
CJNE @RI, #data, rel		if @Ri ≠ data
CJNE A, #data, rel		if A ≠ data
CJNE A, Rx, rel		if A ≠ Rx

EXAMPLE

Calculate the real delay time for the attached nested loop, when $F_{crystal} = 12\text{ MHz}$



```

DELAY: MOV R3, #251    ;1 mc
wait2: MOV R5, #250    ;1 mc
wait1: NOP             ;1 mc
      NOP
      DJNZ R5, wait1    ;2 mc
      DJNZ R3, wait2    ;2 mc
      RET              ;2 mc

```

✓ No. of machine cycles of wait1 = $(1+1+2) \times 250$
= 1000

✓ No. of machine cycles of wait2 = $(1+2+\text{wait1}) \times 251$
= $1003 \times 251 = 251753$

✓ Delay Time = $(1+2+251753) \times \frac{12}{12000000} \approx 0.25 \text{ sec}$

EXAMPLE

Calculate the real run-time for the attached subprogram, when $F_{crystal} = 6\text{ MHz}$

```
DELAY: MOV R3, #25      ;1 mc
wait2: NOP              ;1 mc
        MOV R5, #20
wait1: NOP
        NOP
        NOP
        DJNZ R5, wait1   ;2 mc
        DJNZ R3, wait2
        RET              ;2 mc
```

EXAMPLE

Write assembly code to send the stored data in flash memory “The Microcontroller 8051” Via P2

```
org 0
SJMP main
org 30h
main:
MOV     P2, #0    ; clear Port2
MOV     R0, #0
MOV     DPTR, #My_MSG ;load data pointer with MSG's address

Get_Byte_And_Trans: MOV     A, R0
                   MOVC    A, @A+DPTR
                   MOV     p2, A
                   INC     R0
                   JZ      main      ; if ACC.= 0 goto main
                   SJMP    Get_Byte_And_Trans

;;;;;;;;;; ORG    0100H      Option stored address

My_MSG:   DB      "The Microcontroller 8051", 0
END
```

Explanation

DB "8051", 0

"0" has Machine code = ASCII = 30h

Machine code = 00h

JZ is affected with the machine code 00h